

```
// Programa Ingeniería de Sistemas - Uniamazonia
```

Electiva I - Backend

Fundamentos de JavaScript

Conceptos necesarios

- ✓ var, let y const.
- ✓ mutabilidad
- ✓ funciones
- ✓ callbacks
- ✓ Promesas
- ✓ Then - catch
- ✓ Sincronismo/Asincronismo
- ✓ Async/await

Funciones en JS

Fundamentos de JavaScript



// Declaración de Función

Utiliza la palabra clave `function` seguida de un nombre obligatorio. Sufre de hoisting



```
function sumar(a, b) {  
  return a + b;  
}
```



// Expresión de función

Se crea una función y se asigna como valor a una variable. No se pueden llamar antes de su línea de asignación.



```
const restar = function(a, b) {  
  return a - b;  
};
```

Funciones en JS

Fundamentos de JavaScript



// Funciones Flecha

Introducidas en ES6, ofrecen una sintaxis más corta.



```
const multiplicar= (a, b) => a * b;
```



// Funciones Anónimas

Son funciones que no tienen un nombre asignado. Se usan comúnmente dentro de expresiones `callbacks`



```
setTimeout(function() {  
  console.log("Hola después de 2 segundos");  
}, 2000);
```

| Ejercicios en clase

Calculador de Direcciones IP (S.O. - REDES)

- **Contexto:** Los routers y servidores procesan las direcciones IP en formato binario para calcular máscaras de subred y rutas de manera eficiente.
- **Enunciado:** Crea una función llamada `ipADecimal(ipBinaria)` que reciba una cadena de texto con una dirección IPv4 en formato binario de 32 bits separados por puntos (cuatro octetos de 8 bits cada uno). La función debe retornar la dirección IP tradicional en formato decimal con puntos.
- **Casos de prueba obligatorios** (Se deben validar al menos estos):
 - `ipADecimal("11000000.10101000.00000001.00000001")` → `"192.168.1.1"`
 - `ipADecimal("00001010.00000000.00000000.00000001")` → `"10.0.0.1"`
- **Reto extra:** Validar que cada octeto tenga exactamente 8 caracteres antes de procesar; si no, retornar `"Error: Formato binario inválido"`.

Callbacks

Una función que se pasa como parámetro (argumento) para ser utilizada por otra función

// Crear un Arreglo de enteros

Usando map sobre el arreglo, mostrar el doble de cada uno de sus valores.



```
const arreglo = [1, 2, 3, 4, 5];  
const arreglo2 = arreglo.map(function(elemento) {  
  return elemento * 2;  
});
```

// Crear una función para determinar si un número es par

Pasarla como callback para evaluar cada valor de un array



```
const funcionCallback = (elemento) => {  
  if(elemento % 2 ===(0) {  
    return `${elemento} es par`;  
  } else  
    return `${elemento} es impar`;  
}  
  
const arreglo3 = arreglo.map(funcionCallback);  
console.log(arreglo3);
```

Callbacks

Una función que se pasa como parámetro (argumento) para ser utilizada por otra función

```
// Crear un Arreglo de enteros
```

Usando map sobre el arreglo, mostrar el doble de cada uno de sus valores.



```
const sumar = (a, b) => a + b;
const restar = (a, b) => a - b;
const multiplicar = (a, b) => a * b;
const dividir = (a, b) => a / b;

(
const calculadora = (a, b, operacion) => {
  return operacion(a, b);
}

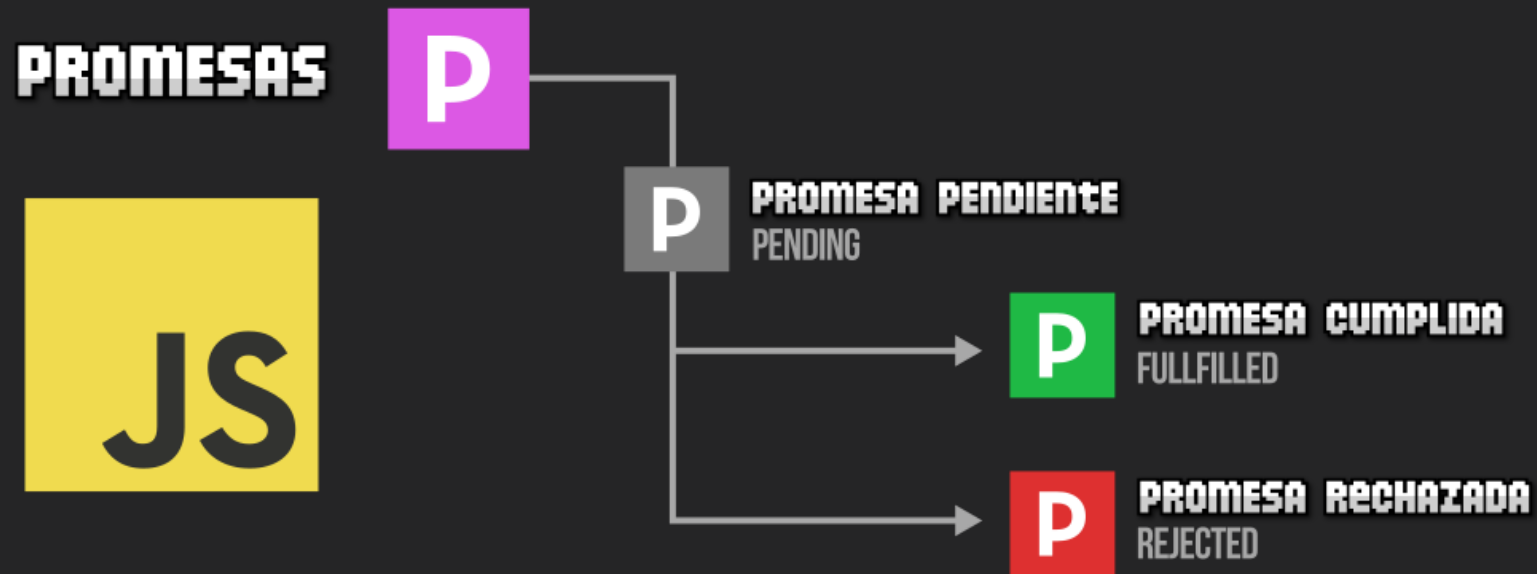
console.log(calculadora(10, 5, sumar));
console.log(calculadora(10, 5, restar));
console.log(calculadora(10, 5, multiplicar));
console.log(calculadora(10, 5, dividir));
```

JS Promises

Fundamentos de JavaScript

Es un objeto especial que nos permitirá **encapsular** una operación, la cual reaccionará a **dos posibles** situaciones dentro de una promesa:

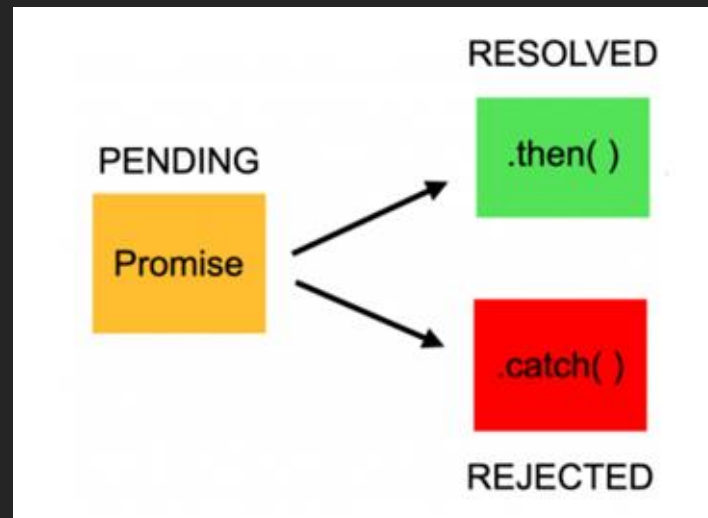
- ✓ ¿Qué debería hacer si la promesa se cumple?
- ✓ ¿Qué debería hacer si la promesa no se cumple?



JS Promises

Fundamentos de JavaScript

Al prometerse algo, es una promesa en estado pendiente (**pending**), no sabemos cuándo se resolverá esa promesa. Sin embargo, cuando llega el momento, se nos notifica si la promesa se cumplió (**Fulfilled**, también lo encontramos como **Resolved**) o tal vez, a pesar del tiempo, al final nos notifiquen que la promesa no pudo cumplirse, se rechazó (**Rejected**).



JS Promises

Fundamentos de JavaScript

Al prometerse algo, es una promesa en estado pendiente (**pending**), no sabemos cuándo se resolverá esa promesa. Sin embargo, cuando llega el momento, se nos notifica si la promesa se cumplió (**Fulfilled**, también lo encontramos como **Resolved**) o tal vez, a pesar del tiempo, al final nos notifiquen que la promesa no pudo cumplirse, se rechazó (**Rejected**).



```
const dividir=(divisor, dividendo) => {  
  return new Promise((resolve, reject) =>  
    if(dividendo === 0) {  
      reject("No se puede dividir entre cero");  
    } else  
      resolve(divisor / dividendo);  
  });  
});
```

JS Promises

Fundamentos de JavaScript



```
const dividir=(divisor, dividendo) => {  
    return new Promise((resolve, reject) =>  
        if(dividendo === 0) {  
            reject("No se puede dividir entre cero");  
        } else  
            resolve(divisor / dividendo);  
    })  
}
```

```
});
```

```
dividir(10, 2).then((resultado) => {  
    console.log(`El resultado es: ${resultado}`);  
}).catch((error) => {  
    console.log(`Error: ${error}`);  
});
```

```
dividir(10, 0).then((resultado) => {  
    console.log(`El resultado es: ${resultado}`);  
}).catch((error) => {  
    console.log(`Error: ${error}`);  
});
```

Sincronismo / Asincronismo

Las operaciones **síncronas** son **bloqueantes**, esto significa que las otras tareas no pueden comenzar a ejecutarse hasta que la primera no haya terminado de ejecutarse.



```
function funcionA(){
    console.log("PRIMERO");
    funcionB();
    console.log("SEGUNDO");
    console.log("TERCERO");
}

function funcionB(){
    console.log("CUARTO");
    funcionC();
    console.log("QUINTO");
}

function funcionC(){
    console.log("SEXTO");
}

funcionA();
```

| Sincronismo / Asincronismo

Async / Await



```
const dividir=(dividendo, divisor) => {  
  return new Promise((resolve, reject) =>  
    if(divisor === 0) {  
      reject("No se puede dividir entre cero");  
    } else  
      resolve(dividendo / divisor);  
  });  
  
});  
  
const funcionAsincrona= async()=>{  
  try  
    {let resultado = await dividir(10, 0);  
    console.log(`El resultado es: ${resultado}`);  
  } catch error) {  
    console.log(`Error: ${error}`);  
  }  
}  
  
funcionAsincrona();
```

Classes



| POO

Objetos/Class



```
const punto1 = new Punto(2, 3, "rojo");  
console.log(punto1.toString()); // (2, 3, rojo)  
  
punto1.mover(1, -2);  
console.log(punto1.toString()); // (3, 1, rojo)
```



```
class Punto{  
  constructor(x, y, color){  
    this.x = x;  
    this.y = y;  
    this.color = color;  
  }  
  
  toString(){  
    return `(${this.x}, ${this.y}, ${this.color})`;  
  }  
  
  mover(dx, dy){  
    this.x += dx;  
    this.y += dy;  
  }  
  
  cambiarColor(nuevoColor){  
    this.color = nuevoColor;  
  }  
}
```



fs module

| Trabajando con Archivos

Módulo FS + Sync



```
import fs from 'fs';  
// const fs= require('fs');  
const filePath= './fileSystem/ejemplo.txt';  
fs.writeFileSync(filePath,"Hi students, This is my first archive");  
if(fs.existsSync(filePath)){  
    let contenido= fs.readFileSync(filePath,'utf8');  
    console.log(contenido);  
    fs.appendFileSync(filePath,"\nadd content");  
    contenido= fs.readFileSync(filePath,'utf8');  
    console.log(contenido);  
    // fs.unlinkSync(filePath); // elimina el archivo  
}else{  
    console.log("No existe el archivo");  
}
```


| Trabajando con Archivos

Módulo FS + promise



```
import fs from 'fs';

const funcionesAsincronas = async () => {
  const filePath= './fileSystem/ejemplo.txt';
  try
    {await fs.promises.writeFile(filePath,"Hi students, This is my first archive");
      console.log("Archivo escrito correctamente");
      const contenido = await fs.promises.readFile(filePath,'utf8');
      console.log(contenido);
      await fs.promises.appendFile(filePath,"\nadd content");
      const contenidoActualizado = await fs.promises.readFile(filePath,'utf8');
      console.log(contenidoActualizado);
      // await fs.promises.unlink(filePath);
      // console.log("Archivo eliminado correctamente");
    } catch err) {
      console.log("Error: ", err);
    }
  };

  funcionesAsincronas();
}
```



| Trabajando con Archivos

Leer un archivo JSON Módulo FS + Promises



```
import fs from 'fs';

const readPackageJson = async () => {
  const filePath= './fsJSON/package-lock.json';
  try
    {const data = await fs.promises.readFile(filePath, 'utf8');
      const packageJson = JSON.parse(data);
      console.log(packageJson);
      console.log(data);
    } catch err) {
      console.log("Error: ", err);
    }
};

readPackageJson();
```



Ejercicios en Clase

ADMINISTRAR USUARIOS CON PERSISTENCIA EN ARCHIVOS

- Crear una Clase para administrar usuarios y trabajar con Archivos JSON
- Usuario: id, nombre, apellido, edad, rol (estudiante, profesor, administrador)
- El id autonumérico
- El constructor debe recibir la ruta del archivo



Ejercicios en Clase

ADMINISTRAR USUARIOS CON PERSISTENCIA EN ARCHIVOS + usuario y password(encriptado)

- Crear una Clase para administrar usuarios y trabajar con Archivos JSON
- Usuario: id, nombre, apellido, edad, rol (estudiante, profesor, administrador), usuario, password
- El id autonumérico
- El constructor debe recibir la ruta del archivo
- El password debe estar encriptado
`return crypto.createHash('sha256').update(password).digest('hex');`



```
async createUser(user){
  if(!user.usuario || !user.password){
    console.error('El usuario y la contraseña son obligatorios');
    return;
  }
  const newUser={
    nombre: user.nombre ?? 'sin nombre',
    apellido: user.apellido ?? 'sin apellido',
    edad: user.edad ?? 'sin edad',
    curso: user.curso ?? 'sin curso',
    usuario: user.usuario ?? 'sin usuario',
    password: this.getHash(user.password ?? 'sin password')
  };
  const users = await this.getAllUsers();
  if(users.some(u => u.nombre === newUser.nombre && u.apellido === newUser.apellido)){
    console.error('El usuario ya existe');
    return;
  }
  users.push(newUser);

  try
  {await fs.promises.writeFile(this.filePath, JSON.stringify(users,null,'\t'));
    console.log('usuario creado correctamente');
  } catch error) {
    console.error("No es posible crear el usuario");
  }
}
```



```
async getAllUsers(){
  try
    {const users = await fs.promises.readFile(this.filePath, 'utf8');
      return JSON.parse(users);
    } catch error) {
      console.log("Error al leer usuarios creados")
      return
    }
  }
}
```



```
getHash(password) {
  return crypto.createHash('sha256').update(password).digest('hex');
}
```